DSA STUDY BLUEPRINT SERIES

48. Knights Tour Problem

Knight's Path visiting every Cell of Chessboard

Section 06 • C++ Core Data Structures & Algorithms

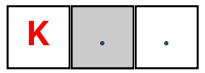
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Move a knight through all $N imes N$ cells without revisiting any.
- Check all 8 potential knight moves and backtrack.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
bool solveKT(int r, int c, int moveIdx, vector>& board) {
    if (moveIdx == n*n) return true;
    for (int i=0; i<8; i++) {
        int nextR = r + xMoves[i], nextC = c + yMoves[i];
        if (isSafe(nextR, nextC, board)) {
            board[nextR][nextC] = moveIdx;
            if (solveKT(nextR, nextC, moveIdx+1, board)) return true;
            board[nextR][nextC] = -1; // Backtrack
        }
    }
    return false;
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(8^{N*N})$ worst case
Space Complexity	$O(N^2)$

Key Practices & Gotchas

- **Important:** Warnsdorff's heuristic optimizes knight move selection to run in linear time.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace backtracking cell navigations in a grid puzzle:

Cell visited	Direction Checked	Move Validation	Dead-end Backtrack step
(0, 0)	Down (D) → (1, 0)	Valid (Open path)	Proceed to cell (1, 0)
(1, 0)	Right (R) → (1, 1)	Invalid (Blocked wall)	Revert to cell (1, 0) and try Next check

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pruning Searches:** Check safety criteria (e.g. diagonal checks for N-Queens) to skip bad board configurations immediately.
- **Visited state maps:** Mark board grid tiles as visited inline to avoid loops, resetting them when returning.
- **Related LeetCode Problems:** #51 N-Queens, #37 Sudoku Solver, #79 Word Search.